Full-stack exercise: an AI-assisted weekly planning SaaS

Starting from **Launchbase**, our open-source monorepo template, build a **weekly planning product** as a real multi-tenant SaaS, with organizations, teams, and an internal admin panel for the people who run it. Model it on how we plan our own weeks at Xaventra. Put an LLM to work inside it with **Pydantic AI** and **Gemini Flash-Lite**, then give it a name, a logo, and a landing page that sells it.

START FROM	EST. EFFORT	AI	BUILD WITH
Launchbase	~12 hours	Pydantic AI + Gemini	Claude Code

01 Start here: name it and give it a logo

Before you write any code, decide what you're launching. This isn't a homework app. It's a product you're putting on the market, so it needs an identity. "Weekly Planner" doesn't count.

Why this comes first. A name is a decision about who the product is for and what it promises them. Everything else hangs off that decision: the line on your landing page, the words on your sign-up screen, the tone of the AI's reviews, even what you choose to build first. Teams already plan in Notion, Linear, and spreadsheets. Why would a lead switch to yours? If you can answer that in one sentence, the name, the logo, and the landing page all get easier, and they'll match each other. Do it now so the brand runs through the code from the first commit instead of being swapped in at the end.

> **We're assessing your entrepreneurial skills here**
> We look for engineers who think like founders, not just ticket-closers. We want to see who you decided the customer is, why they'd care, how you'd get it in front of them, and whether every surface tells the same story. A sharp, honest pitch on a simple product beats a vague one on a clever product.

- **A one-line pitch.** Who it's for and what it does for them, in one sentence. Your landing page hero grows out of this.
- **A brand name.** Short, easy to say, and easy to spell when someone hears it. Check that it isn't already an obvious, well-known product in the same space. Have a sentence ready on why you chose it.
- **A logo, as SVG.** It's used as the **favicon**, in the **app header**, on the **sign-in page**, and on the landing page. It must still read at 16px in a browser tab and on both light and dark backgrounds.

Do this the agentic way too. You don't need a design tool or a designer. Ask your coding agent to brainstorm names against a brief you write, then shortlist and pick one yourself. Have it write the logo as SVG code, render it at 16px, 32px, and large, look at the result, and iterate until it holds up. The judgement is yours, and the agent does the drafting. We care that the logo is clean, deliberate, and works small, not that it looks like a design agency made it.

Carry the brand through all three front ends, replacing Launchbase's: the page `<title>`, the favicon, the header, the sign-in and sign-up screens, the landing page, and the admin panel, which should be clearly marked as internal.

02 Why this exercise

The role is **full-stack**, and this is the shape of a real SaaS product: many organizations sign up, each sees only its own data, and an internal console sits behind it for the team that runs it. More and more of that work also means putting a language model **inside** the product. The model takes structured input, returns structured output, and the app treats that output as data.

This exercise covers all of it in one product you can finish: tenancy and roles, a back office, an AI feature that does real work, and the brand and landing page that put it in front of people. You start from Launchbase, the template our own projects start from, so the wiring is already done. With today's agentic coding tools, one person can build the rest in a few focused days. Most of the skill is in deciding what to build, checking that it actually works, and catching what the agent got wrong.

We picked weekly planning because it's how we work every week. Building a tool for it will teach you a fair amount about how we run things.

03 Start from Launchbase

Launchbase is a monorepo with a FastAPI backend and three TanStack Start front ends. Each part of this exercise has a place in it:

Folder	What it already has	What you build there
`api/`	FastAPI, async SQLAlchemy, Postgres, Alembic, JWT auth, users, pytest. Two surfaces: `/app` and `/internal`.	Sign-up, organizations, teams, plans, the Pydantic AI agents, and the internal endpoints.
`app/`	The customer app: login, an auth-guarded sidebar layout, and a typed API client generated with orval.	The product your customers use.
`internal/`	The staff console, with user management on the `/internal` surface.	The internal admin panel.
`site/`	A statically generated marketing site with a placeholder page and the full shadcn/ui set.	The landing page.

Clone it, then follow the root `README.md` and each folder's `README.md`:

```
git clone https://github.com/chamoda/launchbase your-product-name
cd your-product-name
```

Work with the grain of the template. Launchbase has conventions, and its `AGENTS.md` files spell them out: `uv run` for everything in Python, autogenerated Alembic migrations, no enums in models, services that flush while routers commit, and a typed client you regenerate with `npm run generate` instead of writing fetch calls by hand. Follow them. A new resource should look like the `users` resource already there. Where this brief and an `AGENTS.md` disagree, the brief wins. For example, the template's `AGENTS.md` says tests are optional, but here they're required (see the AI part).

Make the product your own. When you're done, nothing user-facing should still say "Launchbase".

> **Use Linux or macOS. On Windows, use WSL.**
> We develop on Linux (Ubuntu 24.04), and Launchbase's setup steps assume it. Use Linux or macOS. On Windows, install **WSL 2 with Ubuntu** and do everything inside it: your terminal, your coding agent, Postgres, and Node. Keep the repo in the WSL filesystem (under `~/`), not under `/mnt/c`, where file watching and installs are painfully slow. You'll need Python 3.12 with uv, Node.js 24+, Postgres 16, and Docker, which the test suite uses to start its own Postgres.

04 How we plan a week (the domain)

Every engineer publishes a plan at the start of the week. The goal is to **ship one significant thing** by Friday and end the week on a high note, without carrying unfinished work into Monday. A plan is a short list of **tasks**, in three kinds:

Task	What it is
Main task	**One** significant thing to ship this week, and why it matters.
Sub-goals	A few smaller tasks that get the main task over the line.
Go-to-market goals	Tasks that put the work in front of users: analytics events, onboarding, a changelog entry, a landing page update, a feature flag.

Every item is a task, so each one can be ticked off during the week. Good tasks describe a result you can check, not work you'll do: "Checkout handles expired cards, confirmed on staging" is a task you can tick off honestly. "Implement checkout" isn't.

05 The product

Build it the way a real SaaS works: anyone can sign up with an **email and password**, and each customer's data is walled off from everyone else's.

Organizations. Launchbase has login but no sign-up, so add it, using email and password only. A new user signs up and creates an organization, or joins one through an invite. An invite link the owner can copy is fine; you don't need to send email. Each member has a role in the organization: **owner**, **admin**, or **member**. Owners and admins manage members and settings, and members plan their own weeks.

Teams. Admins create teams inside the organization, add members, and give each team a **lead**. A user can belong to more than one team.

Plans. Each member writes a plan for a given week, keyed by that week's **Monday** date: one main task, a few sub-goals, and go-to-market goals. Each is a separate task with its own status, not one free-text blob. A plan moves through **draft → submitted → approved → done**. The team lead **approves** a plan or **sends it back** with a comment. During the week,

tasks are ticked off as **done**, and at the end of the week the member marks whether the main task shipped.

The team's week. A lead picks a week and sees every member at a glance: who has submitted, what's approved, how many of their tasks are done, and whether their goal shipped. A history view shows past weeks.

> **Tenancy is not a UI concern**
> A user in one organization must never be able to read or change another organization's data. The same goes for a member acting as an admin, or a non-lead approving a plan. Enforce all of this **in the API**, not by hiding buttons. When we review, we'll make requests with someone else's IDs and see what comes back.

Seed it. Add a `manage.py` command that creates a believable demo: at least two organizations, a few teams, members in each role, and several weeks of plans, some shipped and some not. We'll explore your product from there, and so should you.

Keep it plain and usable. This isn't a visual design test, but the flows should feel deliberate: loading states while the model thinks, clear empty states, forbidden pages that say so, and no dead buttons.

Out of scope: billing and payments, sending real email (so no email verification or password-reset emails), social login and SSO, GitHub integration, notifications, and deploying it anywhere.

06 The internal admin panel

Build the **back office for the team that runs the product** in `internal/`, on top of the API's `/internal` surface. This is a different thing from an organization's own settings page. A customer's owner or admin must not be able to reach it.

- **Who gets in:** platform staff only. Launchbase already has this: the `is_admin` flag on a user, granted with `uv run manage.py create-user --admin`, and tokens scoped to the internal surface. Keep it that way. Staff are never created through sign-up.
- **Organizations:** every org with its member count, team count, plan count, and when it was last active. Staff can **suspend** an org, which locks its members out until it's reinstated.
- **Users:** search, see which orgs a user belongs to, and deactivate an account.
- **AI usage:** calls per organization, failures, and rate-limited calls, so the team can see what the AI feature costs and where it breaks.

Watch the naming. Launchbase's `is_admin` means **platform staff**, while an organization's **admin** is a customer role. Mixing the two up is the kind of bug that leaks data, so make the difference obvious in your code.

Think about what staff **should** see. Does the back office need the text of customers' plans at all? Make a deliberate call and be ready to explain it.

07 The AI part

Use **Pydantic AI** in your backend, with Google's **Gemini Flash-Lite** model. Build **both** of these:

1. Plan review: "the approval meeting in a box." Before submitting, a member sends their plan to an agent that returns a **structured** review the way a demanding lead would give one. The review flags tasks that describe work to do rather than a result you can check, a main task that doesn't say why it matters, missing go-to-market goals, and scope that doesn't fit in a week. Each finding points at a specific task and comes with a concrete **suggested rewrite**. The UI shows findings next to the tasks they're about, and the member can apply a suggestion with one click.

2. Draft from a brain dump. The member pastes messy notes ("need to finish the billing page, probably also fix that webhook bug, should announce it in the changelog…") and gets back a **pre-filled plan** to edit before saving. The model's output goes into the form. It never goes straight into the database.

> **The AI has to earn its place**
> The model should return **typed data the app acts on**. Use Pydantic models as the agent's `output_type`. Don't ask for JSON in the prompt and then parse a string. If we can swap your AI feature for a textarea and lose nothing, that's the gap this exercise is designed to show.

Getting a key. Go to Google AI Studio and create a free API key. The free tier needs no card. Use the model **`gemini-3.5-flash-lite`**. A minimal agent looks like this:

```python
# cd api && uv add "pydantic-ai-slim[google]"
# GOOGLE_API_KEY=...  in api/.env           (from Google AI Studio)
from pydantic import BaseModel
from pydantic_ai import Agent

class Finding(BaseModel):
    task: str            # which task this is about
    problem: str
    suggested_rewrite: str

class PlanReview(BaseModel):
    verdict: str         # e.g. "approve" | "adjust"
    findings: list[Finding]

reviewer = Agent(
    "google:gemini-3.5-flash-lite",
    output_type=PlanReview,
    instructions="You review weekly engineering plans ...",
)

result = await reviewer.run(plan_as_text)
result.output            # a validated PlanReview, not a string
```

That's the starting point, not the design. Your models, instructions, and how you shape the plan into the prompt are yours to decide, and they're a big part of what we grade.

Requirements for the integration:

- **The key stays on the server.** Read it through the API's settings from `api/.env`. It never reaches the browser and is never committed. Add an empty `GOOGLE_API_KEY=` to `api/.env.example`.
- **The product works without the AI.** If Gemini is slow, down, or returns **429** (the free tier's rate limits are low), planning still works and the UI says what happened in plain words, without hanging or crashing. Don't fire a request on every keystroke.
- **Every call is recorded** against the organization that made it, with whether it succeeded. That's what feeds the admin panel's usage view.
- **Tests don't call Gemini.** Pydantic AI ships `TestModel` and `FunctionModel` for this. `uv run pytest` must pass with no key and no network access to Google, and it should cover your agent code and the tenancy rules, not only the CRUD.

08 The landing page

A product nobody understands doesn't get signups. In `site/`, replace the placeholder with the public page a team lead lands on and decides whether to try the product. We call this **marketing as code**: the page is part of the product, built and shipped like any other feature.

- **Hero:** the name, the logo, one sharp line on what it does and who it's for, and a **Get started** call to action that leads into sign-up in `app/`.

- **How it works:** write the plan, get it approved, track it through the week, explained in your product's own words.
- **Show the AI:** a real example of a plan review, with a vague task flagged and a rewrite suggested. It should be a rendered example, not a stock illustration.
- **For teams:** what a lead gets from it, such as the team's week at a glance, approvals, and seeing who's on track.
- **The finish:** responsive down to a phone, a proper `<title>` and meta description, an Open Graph image so a shared link looks right, and no `lorem ipsum`.

Write the copy yourself or with your agent, but read it out loud before you ship it. Generic AI marketing copy ("Unlock your team's potential") is easy to spot, and we will.

09 Build it with an agentic coding tool

There are two kinds of AI in this exercise. The model **inside** the product is covered above. To **build** the product, we strongly recommend a **full agentic coding tool**, ideally **Claude Code**, or something in the same class like **Codex CLI** or **OpenCode**. These are agents that read your codebase, run commands, write and run tests, and work through a task across many files on their own. Autocomplete-style assistants won't get you through this scope in the time. An agent you direct well will. That's how we build every day, and how fluently you work with one is a large part of what we're hiring for.

Driving it well is the skill:

- **Plan before it codes.** Ask it to propose the data model and the tenancy rules first, review that plan, then let it build.
- **Give it a way to check its own work.** Tests, a seed script, and a running dev server let the agent catch its own mistakes instead of leaving them for you.
- **Point it at the `AGENTS.md` files** so it follows Launchbase's conventions. Claude Code reads `CLAUDE.md`, and a one-line `CLAUDE.md` that says `@AGENTS.md` will load them. Update these files every time the agent gets something wrong, so it doesn't make the same mistake twice.
- **Review everything it hands back.** An agent will happily write an endpoint that returns another organization's plans, or a plan-review prompt that waves "implement checkout" through as a task. **You** have to catch that.

Use it for **everything**, not just the backend: the schema, the tests, the seed data, the admin panel, the brand name, the logo, the landing page copy. Working the fully agentic way means you describe, direct, and review, and the agent types. Treat the agent as a very fast engineer on your team, and yourself as the lead who owns the result. Commit as you go, in small steps.

10 How we review it

What	We look for	Weight
Tenancy & access	No data leaks between orgs. Roles, team leads, and staff-only admin enforced in the API.	★★★
AI integration	Typed outputs the app acts on, reviews worth reading, graceful failure, offline tests.	★★★
Product & domain	Orgs, teams, and the planning week feel like a real product built on how we actually plan.	★★
Admin panel	Useful to the people running the product, with deliberate calls about what staff can see.	★★
Founder thinking	A clear customer and pitch, a name that fits it, a logo that holds up at 16px, and a landing page that sells it.	★★
Code quality	Follows Launchbase's patterns: a clear data model, migrations, a regenerated client, and no dead code the agent left behind.	★
Judgement	Sensible scope calls, and you can tell us what you cut and why.	★

If you have time left over

These are optional and worth less than a solid core. Pick at most one:

- **End-of-week summary.** Given the plan and which tasks were done, draft a short note on what shipped, what didn't, and why.
- **Look back before planning.** Give the agent a **tool** that looks up the member's previous two plans, so it can flag work that keeps carrying over.
- **Team digest.** An AI summary of the team's week for the lead: what's at risk and who hasn't submitted.

11 What to submit

Add a short `SUBMISSION.md` at the repo root covering:

- **How to run it:** anything beyond the Launchbase READMEs, the seed command, and demo logins for a member, a team lead, an org owner, and a staff user.
- **The name:** your brand name, your one-line pitch, and the sentence on why you chose the name.
- **Your hours:** how many hours it actually took you. Be honest; we don't penalise a real number.

What isn’t done: what you cut, what you know isn’t right yet, and what you’d fix with another day. Honesty here scores well.

Then commit everything and package the repo with `git archive`. **Don’t zip the project folder directly.** Between `node_modules` in three front ends, the Python `.venv`, and build output, it runs to gigabytes. `git archive` packs only the files Git tracks, so the zip stays small:

```
git add -A
git commit -m "Final submission"
git archive --format=zip -o assessment.zip HEAD
```

Upload `assessment.zip`.

Check the zip before you send it

`git archive` packs **only what’s committed at `HEAD`**. Uncommitted changes don’t go in, and neither do ignored files like `node_modules`, `.venv`, or your `api/.env`, which keeps your API key out of the zip. Unzip it into a fresh folder and check that the project runs from your instructions before you upload it.